

Técnicas Digitales III

Sistema Operativo

Lucas Liaño

lliano@frba.utn.edu.ar

<https://github.com/lucasliano>

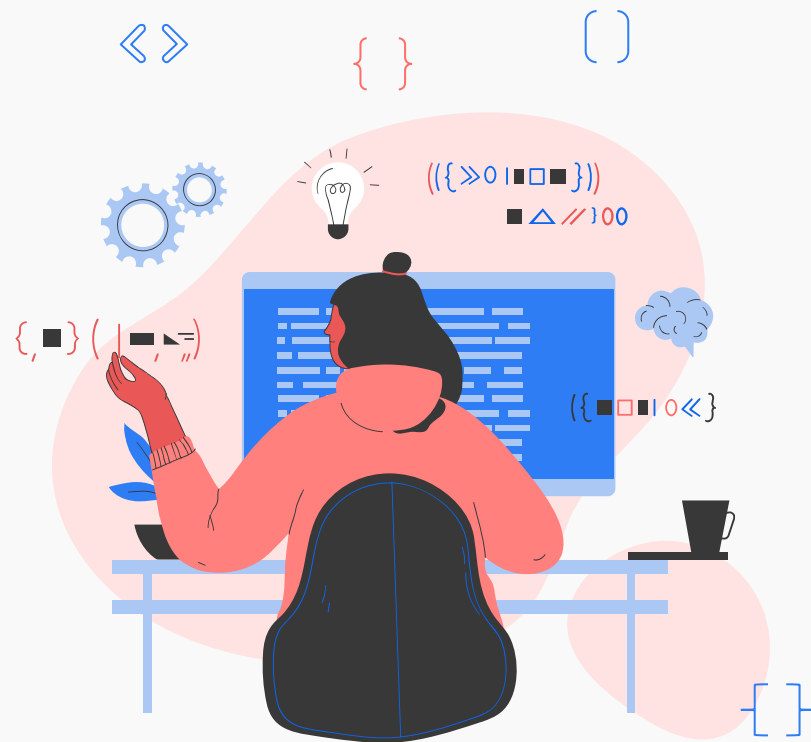


Tabla de Contenidos

01

¿Task?

Alcance de una tarea

03

Scheduling

Descripción de la política de scheduling implementada

02

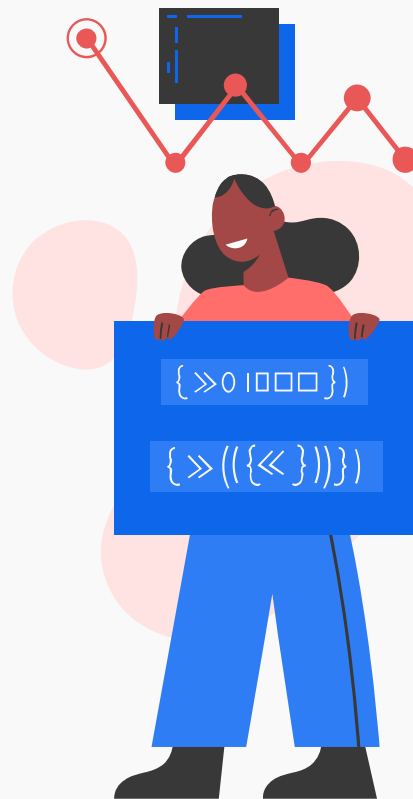
Paginación

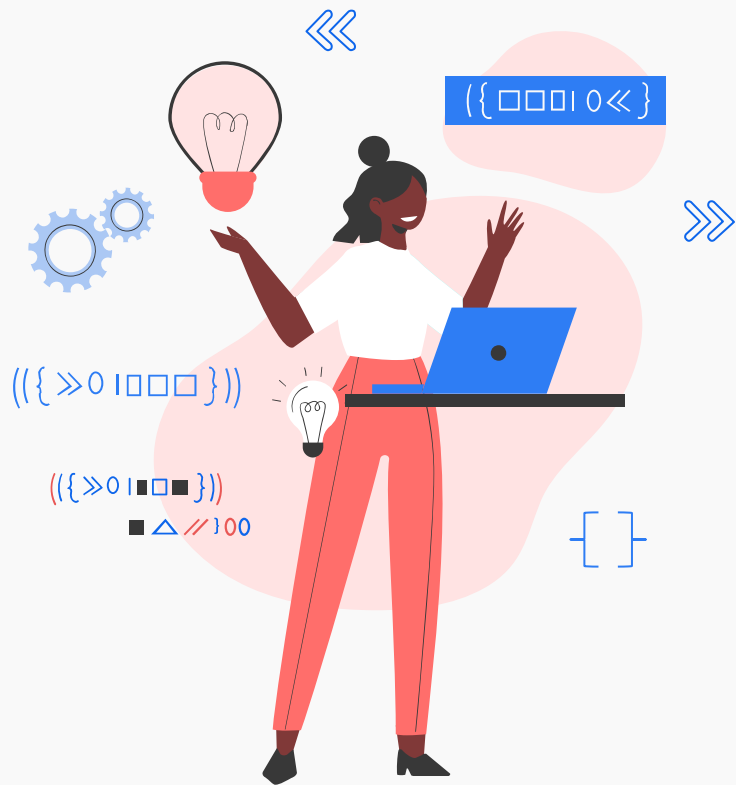
Implementación de la paginación en el scheduler

04

Conclusions

Conclusiones del trabajo



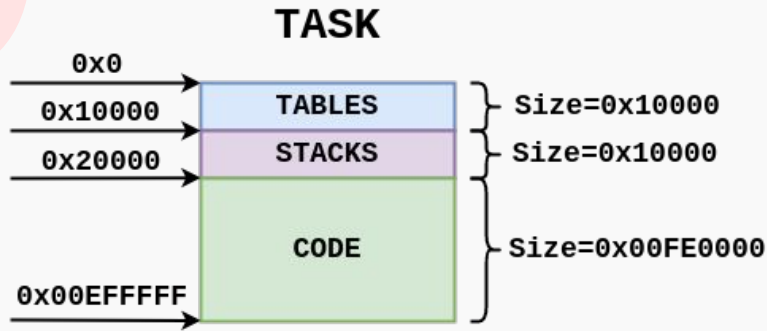


[]

01

¿Task?

{ }



Contenido en Memoria

Una tarea va a almacenar en RAM su código de ejecución, junto con su contexto de stacks. También se reserva una región para poder almacenar las tablas de paginación.

[]

{ }

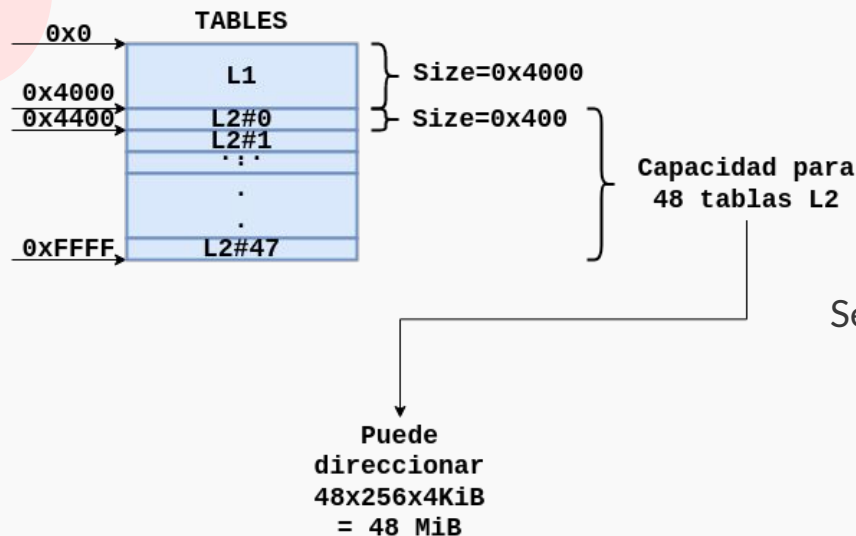
«

»

Tablas de paginación

Se reservó un tamaño total de 4kiB para las tablas de paginación. Esto nos permite paginar un total de 48MiB por tarea.

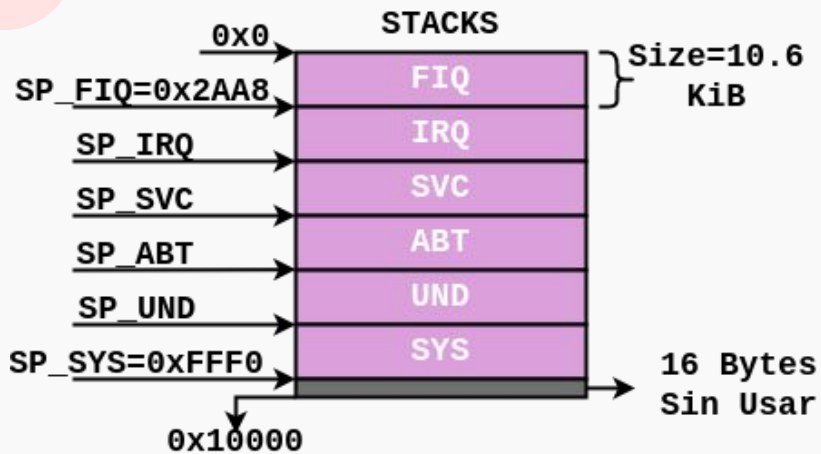
{ }



{ }



[]



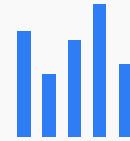
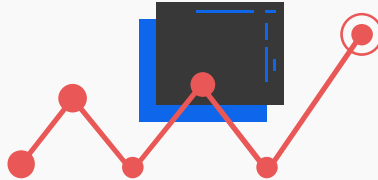
Stacks

La región de stacks se subdivide en distintos stacks, cada uno de aproximadamente 10KiB, para cada modo de operación.

{ }



Sistema Dinámico



`(({{>>0 |□□□}}))`





¿Qué significa que sea dinámico?



Alocación de memoria

Se implementó mediante el handler de **Data Abort**, un sistema que permite asignar RAM a cada tarea



Ejecución de nuevas tareas

Mediante un recurso del sistema, es decir llamando al **SVC**, se puede solicitar la ejecución de una nueva tarea



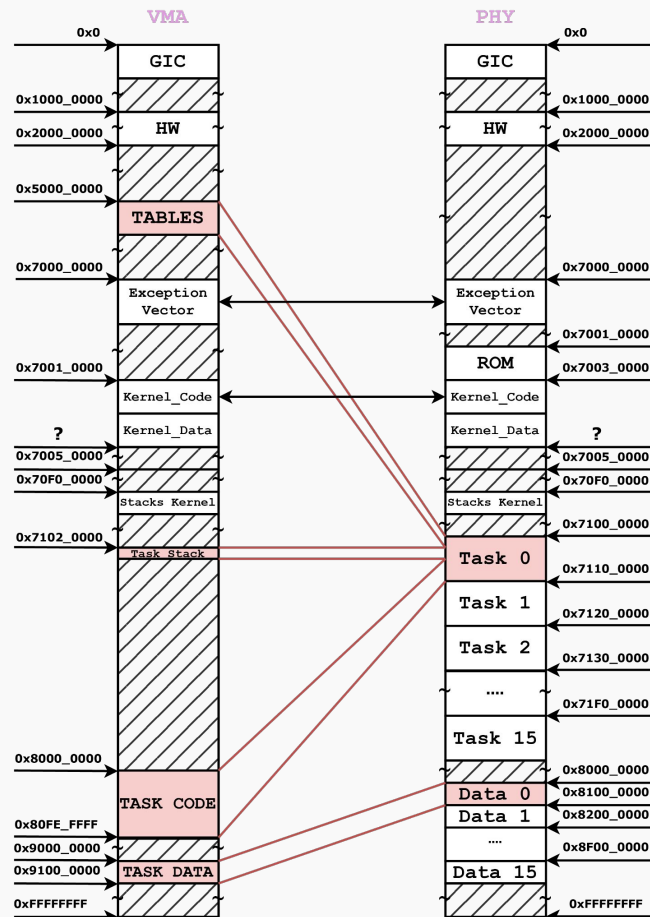


02

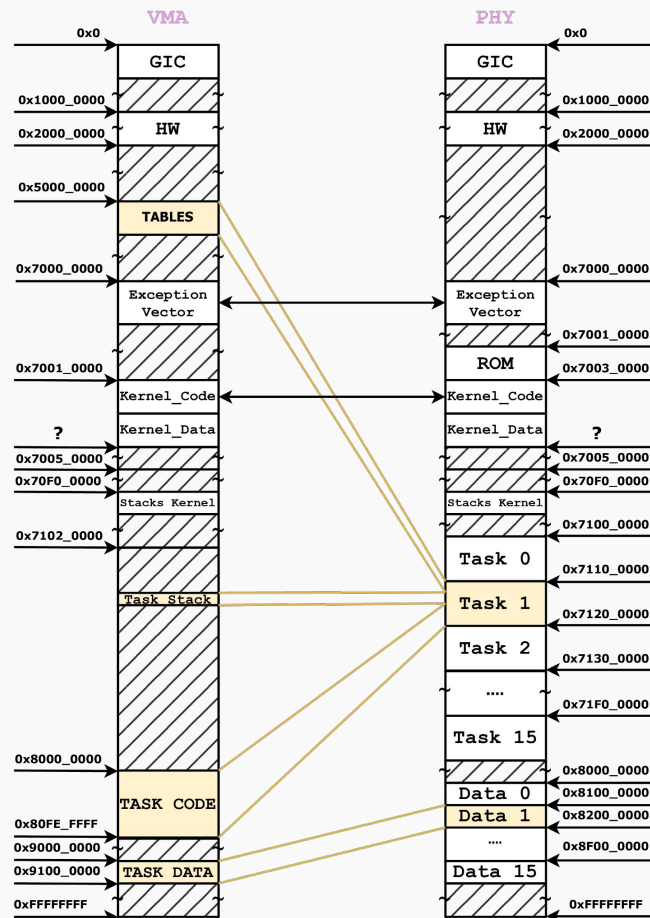
Paginación



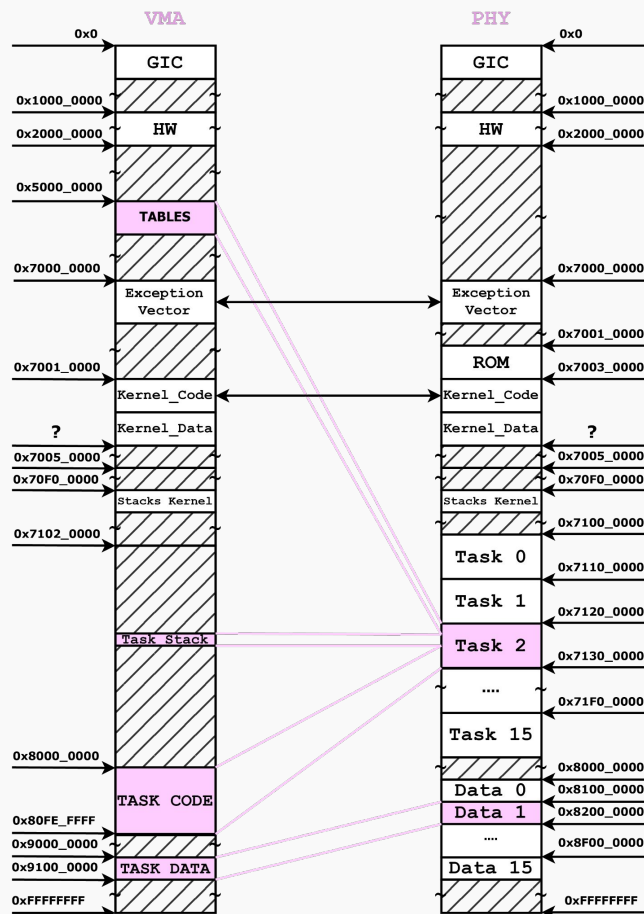
Mapa de Memoria



Mapa de Memoria



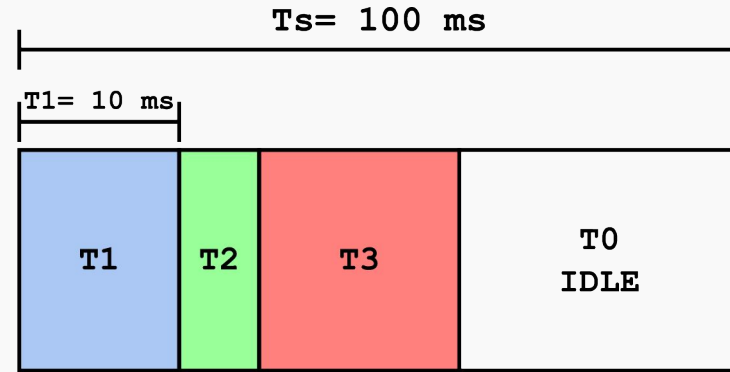
Mapa de Memoria





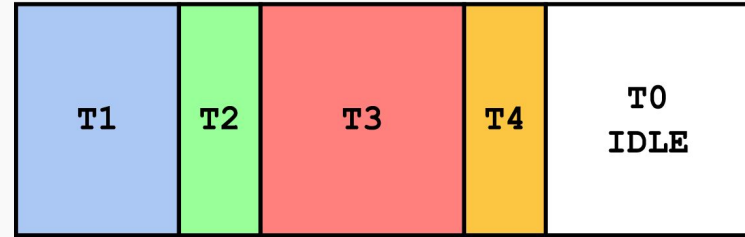
Política de Scheduling

Imaginemos la situación inicial donde tenemos 3 tareas corriendo, junto con la IDLE.



Política de Scheduling

Supongamos que comienza a ejecutarse una nueva tarea. La misma se agrega al final de la lista. También vemos que el tiempo de la tarea IDLE se ajusta al tiempo sobrante en el timeframe.



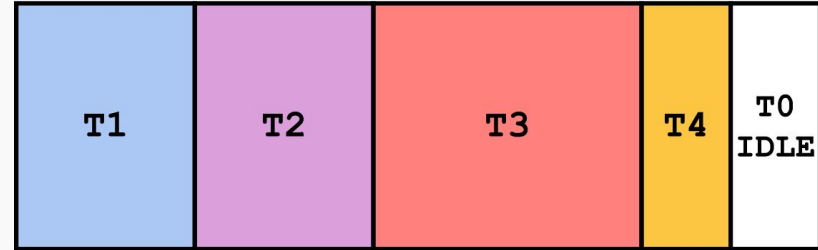
Política de Scheduling

Ahora removimos la tarea 2. Aquí vemos que no se han reasignado los recursos para reorganizar la lista, pero en su lugar se seguirá ejecutando la tarea 3 luego de la 1.



Política de Scheduling

Finalmente, si ahora se lanza una nueva tarea, la misma ocupará el espacio de la tarea 2, a pesar de ser una tarea distinta. Es decir, se reutilizan los PIDs.



Conclusiones



Paginación Realtime

La memoria se asigna en runtime



Protección

Se implementó un sistema de privilegios (USR/SYS)



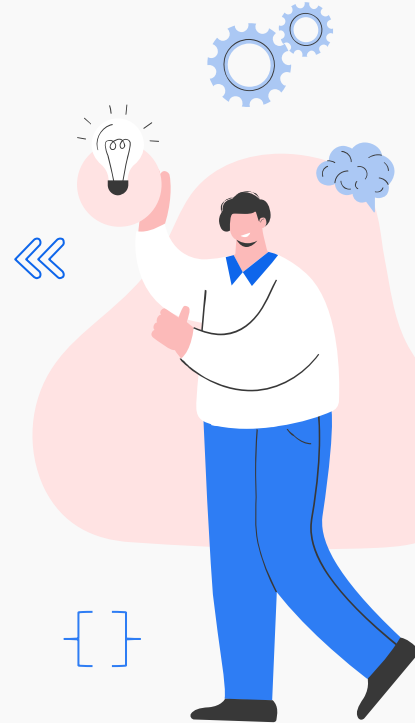
Ejecución Dinámica

Cada tarea puede invocar nuevas tareas en runtime



Scheduling Predecible

El scheduler es capaz de asignar CPU a las distintas maneras en runtime de manera predecible



Thanks!

Do you have any questions?

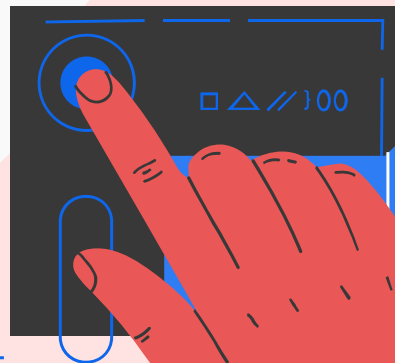
lliano@frba.utn.edu.ar

<https://github.com/lucasliano>

Agradecimiento especial a **Facundo Amor** por ayudarme a diseñar los gráficos.



`{({({ >>}))<<}}`



`(({{>>0i□□□}}))`

```
((: 00 -=>>})  
{ (<1 00 1 000 >>)}  
((: 0)>"< )  
<01 001} +100 0}>  
((: 0)>"< )  
{ (<1 00 1 000 >>)}
```

